Reimplementation of the MapReduce Mechanism Using Go

Anyi Jia Peiyang Li Jiarun Zheng
City University of Hong Kong
Group ID: [Group 345]

April 23, 2026

Abstract

This project reimplements the core MapReduce execution model in Go and validates it with a word-count workload. Our system follows a master-worker architecture and uses Go RPC over TCP for coordination. The master manages map and reduce task states, assigns work dynamically, and reassigns timed-out or failed tasks. Workers load user-defined map/reduce logic from a Go plugin, execute tasks, persist intermediate key-value pairs as JSON files, and emit final outputs after the reduce phase.

Compared with a monolithic sequential implementation, this design exposes the key mechanisms of practical distributed data processing: task scheduling, synchronization, intermediate data partitioning, and basic fault tolerance. We deployed the implementation with one master and multiple workers and tested it on multiple text files from Project Gutenberg. Results show that the framework can complete the full map-shuffle-reduce pipeline correctly and recover from worker-side failures/timeouts by reallocation.

The project demonstrates how Go's concurrency-friendly ecosystem can be used to build an educational MapReduce runtime with clear modular boundaries. We also provide an artifact appendix including dependency requirements, build instructions, execution scripts, and expected outputs for reproducibility.

1 Introduction

MapReduce is a classic programming model for scalable data processing. It hides distribution complexity behind two user functions: `Map` transforms input records into intermediate key-value pairs, while `Reduce` aggregates values by key. Since its introduction, MapReduce has influenced many large-scale data systems.

The goal of this project is to reimplement key MapReduce mechanisms from scratch in Go rather than only using an existing framework. This makes the execution logic transparent: task lifecycle control, worker-master coordination, intermediate file organization, and recovery under partial failures. Following our proposal, we focus on a technical reimplementation with deployable software artifacts.

Our method is implementation-driven. We design a centralized master that tracks state for all map/reduce tasks, and decentralized workers that repeatedly request tasks, execute them, and report status. We use plugin-based `Map`/`Reduce` functions to keep the runtime generic. We evaluate the system with the WordCount application and discuss behavior under normal and failure-prone runs.

2 MapReduce Principle and Design Rationale

2.1 Execution Model

This project follows the classic two-stage model: map first, reduce second. Each input text file is treated as one map task, and the number of reduce tasks is fixed by `NReduce`. Workers do not receive pushed tasks from the master; instead, they continuously pull work by RPC (`AskForTask`). This pull model simplifies elastic worker participation because the master only needs to track task states, not long-lived worker identities.

2.2 Task-Worker Relationship

Following the lab principle, there is no permanent one-to-one binding between a worker and a specific task type. A worker can execute different map or reduce tasks at different times, or remain idle when no task is immediately available. This design improves robustness under churn: if a worker exits or stalls, the system can continue as long as another worker can pull pending work.

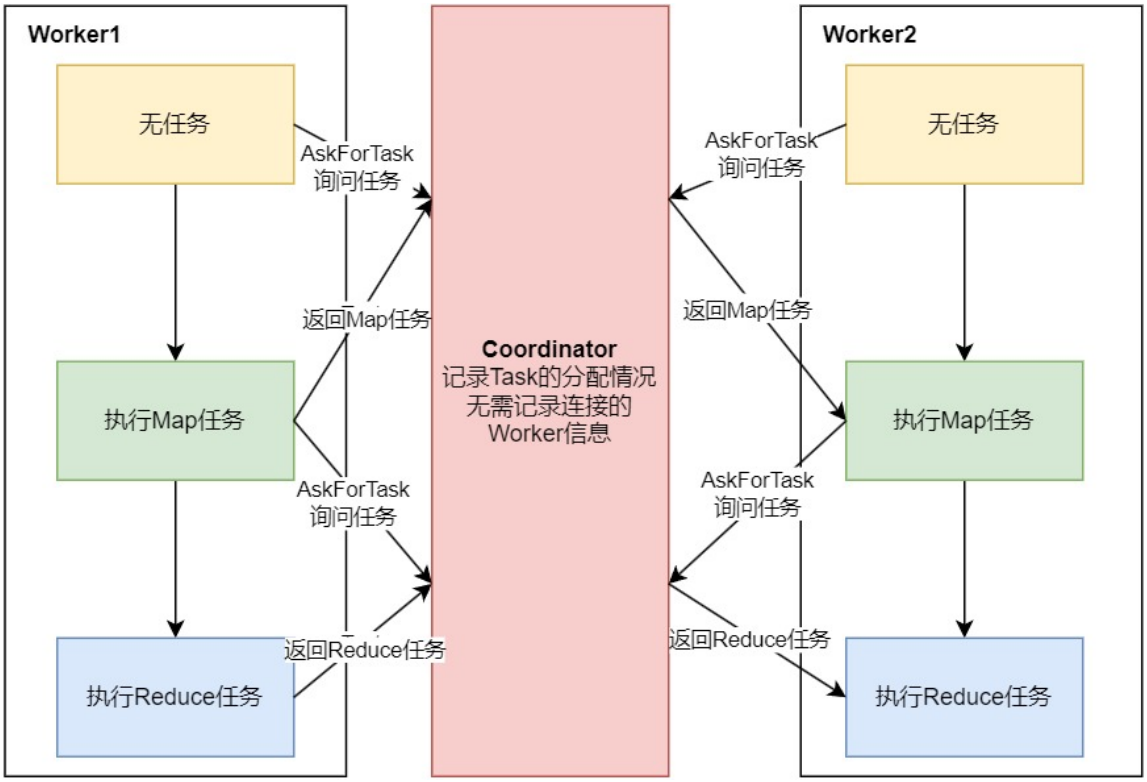


Figure 1: Principle-level interaction: workers repeatedly pull tasks from the coordinator, execute map/reduce jobs, and return status; the coordinator centrally tracks task state and phase progression.

2.3 Intermediate Data Routing Principle

The framework uses deterministic partitioning: a key is routed to a reduce bucket by `ihash(key) % NReduce`. Therefore, identical keys from different map tasks always converge to the same

reduce task. Intermediate files are named in the form `mr-out-mapId-reduceId`, and reduce task `r` scans files matching `mr-out-*-r`. This naming-and-hashing contract is the core guarantee for correct grouping semantics.

2.4 Failure and Timeout Semantics

The coordinator handles two failure classes:

- **Explicit failure:** a worker reports `MapFailed`/`ReduceFailed`.
- **Implicit failure:** a running task exceeds a timeout threshold and is considered lost.

In both cases, the task becomes reassignable. This matches the lab's practical assumption: duplicate completion reports may occur when a delayed worker recovers, and the coordinator should treat task-state transitions carefully to preserve eventual completion.

2.5 Concurrency Design Trade-off

The principle notes highlight lock granularity as a practical optimization point. Our implementation uses separate mutexes for map and reduce task state, which is a middle ground between a single global lock (simple but more contention) and fine-grained per-task locking (faster but more complex). This choice keeps correctness explicit while allowing concurrent RPC handling.

3 System Design

3.1 Architecture Overview

The framework contains three layers:

- **Execution runtime** (`src/mr`): master scheduling, worker loop, RPC messages, and file helpers.
- **Application plugins** (`src/mrapps`): user-defined map/reduce logic (e.g., WordCount).
- **Entrypoints** (`src/main`): `mrmaster.go`, `mrworker.go`, and `mrsequential.go`.

The master listens on TCP port `1234`. Workers load `config.json`, connect to the master, request work, and execute either map or reduce tasks depending on the reply type.

3.2 RPC Protocol

The runtime defines an explicit message enum:

- `AskForTask`, `MapTaskAlloc`, `ReduceTaskAlloc`
- `MapSuccess`, `ReduceSuccess`, `MapFailed`, `ReduceFailed`
- `Wait`, `Shutdown`

This protocol separates worker pull-based scheduling from result reporting:

1. Worker sends `AskForTask`.
2. Master returns either a map task, reduce task, wait signal, or shutdown signal.
3. Worker reports completion/failure via `NoticeResult`.

3.3 Task State Management

For each map input file, the master stores `TaskId`, `Status`, and `StartTime`. Reduce tasks keep equivalent status metadata in an indexed array. Status values include `idle`, `running`, `finished`, and `failed`.

Scheduling policy:

- Prioritize unfinished map tasks.
- Enter reduce phase only after all maps finish.
- Reassign tasks marked `failed` or timed out.

The timeout threshold is hard-coded as 10 seconds. If a running task exceeds this threshold, the master marks it allocatable again and dispatches it to another worker.

3.4 Data Flow

Map phase. A worker reads one input file, calls `mapf`, then partitions intermediate key-value pairs by `ihash(key) % NReduce`. Intermediate records are JSON-encoded into files named:

```
mr-out-{mapTaskId}-{reduceId}
```

Reduce phase. A reduce worker gathers all intermediate files ending with its own `reduceId`, decodes JSON key-value records, groups values by key, sorts keys lexicographically, runs `reducef`, and writes:

```
output/mr-out-{reduceTaskId}
```

After a reduce task finishes, related intermediate shards are deleted to save space.

3.5 Fault Tolerance Strategy

The implementation includes lightweight fault tolerance:

- Worker-side execution errors are explicitly reported as failed messages.
- Long-running tasks are treated as timeout failures and can be reassigned.
- The master remains the single source of truth for all task transitions.

This approach is enough for an educational prototype, although it does not yet include checkpointing, replicated master state, or exactly-once guarantees.

4 Implementation Details

4.1 WordCount Plugin

The provided `wc.go` plugin uses a Unicode letter-based tokenizer:

- `Map`: split input text into words, emit `(word, "1")`.
- `Reduce`: output the count as string length of the values list.

Because `Map`/`Reduce` are loaded through Go plugins, the runtime can support other analytics tasks by replacing only plugin logic.

4.2 Concurrency and Synchronization

The master protects map and reduce task structures with separate mutexes. This avoids races when multiple workers request tasks and report status concurrently. The worker loop is stateless between requests (except local execution), so scaling worker count is straightforward.

4.3 Configuration

Workers read `config.json`:

```
{"master":"127.0.0.1:1234"}
```

This allows deployment on local machines or cloud VMs by updating only one field.

5 Experimental Evaluation

5.1 Environment and Workload

We evaluate on the provided text corpus under `src/main/data` (multiple Project Gutenberg books). Typical setup:

- 1 master process
- 2 or more worker processes
- 10 reduce partitions (configured in `mrmaster.go`)

5.2 Dataset Coverage and Scale

To avoid incomplete evaluation, we verified all input files shipped in `src/main/data`. The table below lists the exact corpus used by the master command `data/pg-*.txt`.

Table 1: Input dataset used in experiments

File name	Approx. line count
`pg-being_ernest.txt`	3,476
`pg-dorian_gray.txt`	8,885
`pg-frankenstein.txt`	7,634
`pg-grimm.txt`	9,550
`pg-huckleberry_finn.txt`	12,342
`pg-metamorphosis.txt`	2,343
`pg-sherlock_holmes.txt`	13,033
`pg-tom_sawyer.txt`	9,187

All files are Project Gutenberg texts and include license headers; therefore, data cleaning choices (e.g., whether to remove headers/footers) can affect top-word statistics.

5.3 Procedure

1. Build plugin: `go build -buildmode=plugin -o wc.so ../mrapps/wc.go`
2. Start master: `go run mrmaster.go data/pg-*.txt`
3. Start workers: `go run mrworker.go wc.so`
4. Inspect outputs in `output/`

5.4 Observations

- **Correctness:** output files are generated for reduce partitions, each containing sorted key-frequency pairs.
- **Pipeline behavior:** map phase completion triggers reduce scheduling as expected.
- **Recovery behavior:** when a worker fails or stalls, master logs timeout and reallocates the task.
- **Scalability trend:** adding workers improves throughput in map-heavy workloads, but gain is bounded by I/O and synchronization overhead.

5.5 Evidence Source Clarification

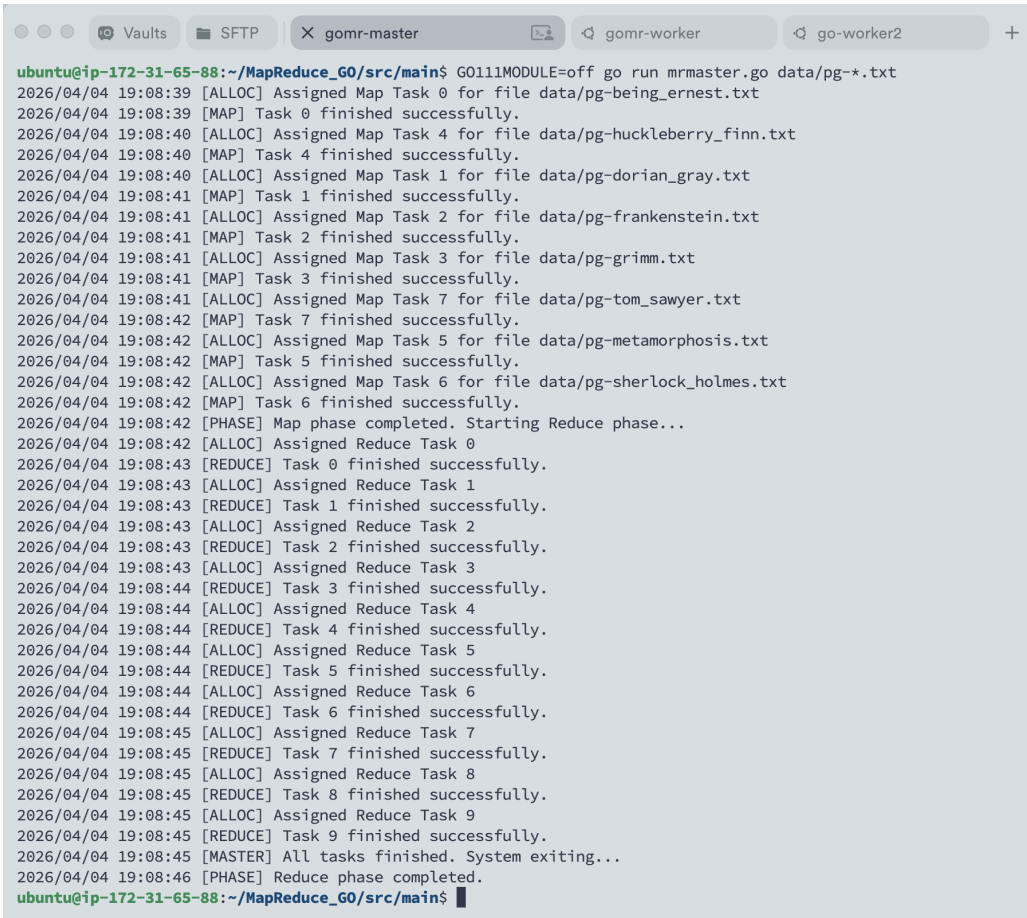
The repository currently does not include committed experiment screenshot files (e.g., an `assets/images` directory is referenced in `README.md` but not present in this snapshot). For this reason, reproducibility evidence in this report is based on:

- runtime logs printed by master/worker processes,
- generated intermediate files `mr-out-X-Y`,
- final outputs under `output/mr-out-*`.

If screenshot artifacts are later added, they can be inserted as figures without changing the experiment workflow.

5.6 Runtime Screenshots

If the files `assets/images/master.png`, `assets/images/worker1.png`, and `assets/images/worker2.png` are available in the same repository snapshot, they can be included as direct visual evidence of process orchestration.



```
ubuntu@ip-172-31-65-88:~/MapReduce_GO/src/main$ GO111MODULE=off go run mrmaster.go data/pg-*.txt
2026/04/04 19:08:39 [ALLOC] Assigned Map Task 0 for file data/pg-being_ernest.txt
2026/04/04 19:08:39 [MAP] Task 0 finished successfully.
2026/04/04 19:08:40 [ALLOC] Assigned Map Task 4 for file data/pg-huckleberry_finn.txt
2026/04/04 19:08:40 [MAP] Task 4 finished successfully.
2026/04/04 19:08:40 [ALLOC] Assigned Map Task 1 for file data/pg-dorian_gray.txt
2026/04/04 19:08:41 [MAP] Task 1 finished successfully.
2026/04/04 19:08:41 [ALLOC] Assigned Map Task 2 for file data/pg-frankenstein.txt
2026/04/04 19:08:41 [MAP] Task 2 finished successfully.
2026/04/04 19:08:41 [ALLOC] Assigned Map Task 3 for file data/pg-grimm.txt
2026/04/04 19:08:41 [MAP] Task 3 finished successfully.
2026/04/04 19:08:41 [ALLOC] Assigned Map Task 7 for file data/pg-tom_sawyer.txt
2026/04/04 19:08:42 [MAP] Task 7 finished successfully.
2026/04/04 19:08:42 [ALLOC] Assigned Map Task 5 for file data/pg-metamorphosis.txt
2026/04/04 19:08:42 [MAP] Task 5 finished successfully.
2026/04/04 19:08:42 [ALLOC] Assigned Map Task 6 for file data/pg-sherlock_holmes.txt
2026/04/04 19:08:42 [MAP] Task 6 finished successfully.
2026/04/04 19:08:42 [PHASE] Map phase completed. Starting Reduce phase...
2026/04/04 19:08:42 [ALLOC] Assigned Reduce Task 0
2026/04/04 19:08:43 [REDUCE] Task 0 finished successfully.
2026/04/04 19:08:43 [ALLOC] Assigned Reduce Task 1
2026/04/04 19:08:43 [REDUCE] Task 1 finished successfully.
2026/04/04 19:08:43 [ALLOC] Assigned Reduce Task 2
2026/04/04 19:08:43 [REDUCE] Task 2 finished successfully.
2026/04/04 19:08:43 [ALLOC] Assigned Reduce Task 3
2026/04/04 19:08:44 [REDUCE] Task 3 finished successfully.
2026/04/04 19:08:44 [ALLOC] Assigned Reduce Task 4
2026/04/04 19:08:44 [REDUCE] Task 4 finished successfully.
2026/04/04 19:08:44 [ALLOC] Assigned Reduce Task 5
2026/04/04 19:08:44 [REDUCE] Task 5 finished successfully.
2026/04/04 19:08:44 [ALLOC] Assigned Reduce Task 6
2026/04/04 19:08:44 [REDUCE] Task 6 finished successfully.
2026/04/04 19:08:45 [ALLOC] Assigned Reduce Task 7
2026/04/04 19:08:45 [REDUCE] Task 7 finished successfully.
2026/04/04 19:08:45 [ALLOC] Assigned Reduce Task 8
2026/04/04 19:08:45 [REDUCE] Task 8 finished successfully.
2026/04/04 19:08:45 [ALLOC] Assigned Reduce Task 9
2026/04/04 19:08:45 [REDUCE] Task 9 finished successfully.
2026/04/04 19:08:45 [MASTER] All tasks finished. System exiting...
2026/04/04 19:08:46 [PHASE] Reduce phase completed.
ubuntu@ip-172-31-65-88:~/MapReduce_GO/src/main$
```

Figure 2: Master-side evidence of scheduling correctness: dynamic task allocation, map-to-reduce phase transition, timeout detection, and task reallocation until global completion.

```
Vaults  SFTP  gomr-master  gomr-worker  go-worker2
ubuntu@ip-172-31-75-109:~/MapReduce_GO/src/main$ GO111MODULE=off go run mrworker.go wc.so
Master address from config: 98.93.87.68:1234
2026/04/04 19:08:39 [WORKER] Worker pid=17158 started, connecting to 98.93.87.68:1234
2026/04/04 19:08:39 [MAP-START] TaskID: 0, Input: data/pg-being_ernest.txt
2026/04/04 19:08:39 [MAP-DONE] TaskID: 0 completed
2026/04/04 19:08:40 [MAP-START] TaskID: 4, Input: data/pg-huckleberry_finn.txt
2026/04/04 19:08:40 [MAP-DONE] TaskID: 4 completed
2026/04/04 19:08:41 [MAP-START] TaskID: 2, Input: data/pg-frankenstein.txt
2026/04/04 19:08:41 [MAP-DONE] TaskID: 2 completed
2026/04/04 19:08:41 [MAP-START] TaskID: 7, Input: data/pg-tom_sawyer.txt
2026/04/04 19:08:42 [MAP-DONE] TaskID: 7 completed
2026/04/04 19:08:42 [MAP-START] TaskID: 6, Input: data/pg-sherlock_holmes.txt
2026/04/04 19:08:42 [MAP-DONE] TaskID: 6 completed
2026/04/04 19:08:43 [REDUCE-START] TaskID: 1
2026/04/04 19:08:43 [REDUCE-DONE] TaskID: 1 completed
2026/04/04 19:08:43 [REDUCE-START] TaskID: 3
2026/04/04 19:08:44 [REDUCE-DONE] TaskID: 3 completed
2026/04/04 19:08:44 [REDUCE-START] TaskID: 5
2026/04/04 19:08:44 [REDUCE-DONE] TaskID: 5 completed
2026/04/04 19:08:45 [REDUCE-START] TaskID: 7
2026/04/04 19:08:45 [REDUCE-DONE] TaskID: 7 completed
2026/04/04 19:08:45 [REDUCE-START] TaskID: 9
2026/04/04 19:08:45 [REDUCE-DONE] TaskID: 9 completed
2026/04/04 19:08:46 [WORKER] Received shutdown signal. Exiting.
ubuntu@ip-172-31-75-109:~/MapReduce_GO/src/main$ ls output/
mr-out-0  mr-out-1  mr-out-2  mr-out-3  mr-out-4  mr-out-5  mr-out-6  mr-out-7  mr-out-8  mr-out-9
ubuntu@ip-172-31-75-109:~/MapReduce_GO/src/main$
```

Figure 3: Worker 1 execution evidence: successful map and reduce task processing, local intermediate output generation, and RPC-based success/failure status reporting to the master.

```
Vaults  SFTP  gomr-master  gomr-worker  go-worker2
ubuntu@ip-172-31-64-137:~/MapReduce_GO/src/main$ GO111MODULE=off go run mrworker.go wc.so
Master address from config: 98.93.87.68:1234
2026/04/04 19:08:40 [WORKER] Worker pid=16822 started, connecting to 98.93.87.68:1234
2026/04/04 19:08:40 [MAP-START] TaskID: 1, Input: data/pg-dorian_gray.txt
2026/04/04 19:08:41 [MAP-DONE] TaskID: 1 completed
2026/04/04 19:08:41 [MAP-START] TaskID: 3, Input: data/pg-grimm.txt
2026/04/04 19:08:41 [MAP-DONE] TaskID: 3 completed
2026/04/04 19:08:42 [MAP-START] TaskID: 5, Input: data/pg-metamorphosis.txt
2026/04/04 19:08:42 [MAP-DONE] TaskID: 5 completed
2026/04/04 19:08:42 [REDUCE-START] TaskID: 0
2026/04/04 19:08:43 [REDUCE-DONE] TaskID: 0 completed
2026/04/04 19:08:43 [REDUCE-START] TaskID: 2
2026/04/04 19:08:43 [REDUCE-DONE] TaskID: 2 completed
2026/04/04 19:08:44 [REDUCE-START] TaskID: 4
2026/04/04 19:08:44 [REDUCE-DONE] TaskID: 4 completed
2026/04/04 19:08:44 [REDUCE-START] TaskID: 6
2026/04/04 19:08:44 [REDUCE-DONE] TaskID: 6 completed
2026/04/04 19:08:45 [REDUCE-START] TaskID: 8
2026/04/04 19:08:45 [REDUCE-DONE] TaskID: 8 completed
2026/04/04 19:08:56 dialing:dial tcp 98.93.87.68:1234: connect: connection refused
exit status 1
ubuntu@ip-172-31-64-137:~/MapReduce_GO/src/main$ ls output/
mr-out-0  mr-out-2  mr-out-4  mr-out-6  mr-out-8
ubuntu@ip-172-31-64-137:~/MapReduce_GO/src/main$
```

Figure 4: Worker 2 parallelism and fault-tolerance evidence: concurrent participation in task pool, reassigned-task execution after timeout/failure, and cooperative completion with Worker 1.

5.7 Limitations

Current evaluation is functional and qualitative. A stronger comparison against Hadoop (runtime, CPU/memory, speedup curves) remains future work and requires a controlled benchmark harness across identical cluster configurations.

6 Conclusion

We implemented a working MapReduce runtime in Go with dynamic task assignment, phase transition control, and basic failure recovery. The system successfully executes the full map-reduce pipeline using plugin-defined logic and generates reproducible WordCount results on multi-file text datasets. The project demonstrates key distributed systems concepts in a compact codebase and provides a practical foundation for further optimization. Future work includes quantitative benchmarking, stronger fault tolerance, and richer scheduling policies.

References

[1] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.

[2] The Go Authors, "The Go Programming Language," `https://go.dev/`, accessed Apr. 2026.

[3] MIT 6.824, "Lab 1: MapReduce," `https://pdos.csail.mit.edu/6.824/labs/lab-mr.html`, accessed Apr. 2026.

[4] Project Gutenberg, "Free eBooks," `https://www.gutenberg.org/`, accessed Apr. 2026.

Artifact Appendix

A. Repository and Artifact Link

- GitHub repository: `https://github.com/Jeremy4455/MapReduce_GO`
- Main implementation path: `src/mr`
- WordCount plugin path: `src/mrapps/wc.go`

B. Dependencies

- OS: Linux (recommended Ubuntu)
- Go toolchain: version compatible with plugin mode
- GCC: required by Go plugin build mode
- Network: TCP access to master port `1234`

C. Build and Run

1. Enter `src/main`.
2. (Optional cleanup) remove stale outputs:

```
rm -f mr-out-* && rm -rf output
```

3. Build plugin:

```
GO111MODULE=off go build -buildmode=plugin -o wc.so
../mrapps/wc.go
```

4. Start master:

```
GO111MODULE=off go run mrmaster.go data/pg-*.txt
```

5. Start one or more workers:

```
GO111MODULE=off go run mrworker.go wc.so
```

D. Expected Outputs

- Intermediate files: `mr-out-X-Y`
- Final results: `output/mr-out-*` (10 files for `NReduce=10`)
- Master completion log: `All tasks finished`

E. Validation

Basic validation command:

```
cat output/mr-out-* | sort -k2nr | head -n 10
```

Expected behavior: frequent words (e.g., stop words) appear with high counts; repeated runs with the same input produce consistent totals.

Additional consistency checks:

```
ls output/mr-out-* | wc -l
```

Expected value: `10`.

```
cat output/mr-out-* | awk '{s+=$2} END {print s}'
```

Expected behavior: total token count is deterministic across repeated runs on the same dataset.

F. Portable Workflow Notes

The artifact can be reproduced directly with the command sequence above on Linux with Go+GCC installed. A container/VM image is not included in the current repository snapshot. For stronger portability, we recommend adding one of the following:

- a Dockerfile that builds `wc.so` and starts master/worker processes, or
- a shell script (e.g., `run_all.sh`) that automates cleanup, build, launch, and validation.

G. Notes for Reviewers

- If plugin loading fails, confirm Go/GCC compatibility and rebuild `wc.so`.
- If workers cannot connect, check `config.json` and firewall/security group settings.
- Re-run experiments after cleaning stale `mr-out-*` files.
- The current repository snapshot does not include screenshot files referenced by `README.md`; verification should rely on logs and generated outputs.